Entornos reproducibles para la formación en física: arquitectura, opciones de despliegue y protocolo de evaluación

Reproducible environments for physics education: architecture, deployment options and evaluation protocol

Arturo Sánchez Pineda[1,2]

[1]Creative Commons Venezuela
[2]Creative Commons Switzerland

Correspondencia: Arturo Sánchez Pineda; me@arturos.ch

Resumen. La formación práctica en física necesita entornos que permitan ejecutar análisis, revisar decisiones y conservar resultados. Este ensayo técnico presenta una arquitectura basada en Git, contenedores de desarrollo, cuadernos y automatización. Mediante análisis documental se comparan Codespaces, Binder, JupyterHub y Colab según administración, persistencia y restricciones operativas, en el contexto de materiales docentes latinoamericanos. Los resultados distinguen disponibilidad del repositorio, ejecución verificable y evaluación de una sesión como objetos diferentes. Se propone un protocolo que separa preparación docente, incorporación del estudiante, consumo de recursos e interpretación científica, acompañado de plantillas de registro. El análisis muestra cómo seleccionar una infraestructura según la tarea y el soporte disponible, y cómo transferir prácticas reproducibles a otras actividades. El repositorio editorial proporciona un ejemplo ejecutable del mismo procedimiento.

Palabras clave: computación científica; física; reproducibilidad; Git; automatización; formación.

Abstract. Practical physics education requires environments that support analysis, review of decisions, and preservation of results. This technical essay presents an architecture based on Git, development containers, notebooks, and automation. Documentary analysis compares Codespaces, Binder, JupyterHub, and Colab in terms of administration, persistence, and operational constraints, contextualized in Latin American teaching materials. Results distinguish repository availability, verifiable execution, and session evaluation as separate objects. A proposed protocol separates instructor preparation, student onboarding, resource consumption, and scientific interpretation, supported by recording templates. The analysis explains how infrastructure can be selected according to the task and available support, and how reproducible practices can transfer to other activities. The editorial repository provides an executable example of the same procedure.

Keywords: scientific computing; physics; reproducibility; Git; automation; education.

1. Introducción

En una sesión práctica de física, el tiempo disponible debe permitir formular preguntas, ejecutar análisis e interpretar resultados. Sin embargo, la diversidad de sistemas operativos, bibliotecas y permisos puede convertir el inicio de la clase en una serie de instalaciones individuales. El

problema no se limita a comodidad técnica: si cada estudiante parte de un entorno distinto, resulta más difícil distinguir un error conceptual de uno causado por la configuración.

Las buenas prácticas de computación científica relacionan organización de archivos, control de versiones y automatización con la posibilidad de comprender y repetir el trabajo [1]. Incorporarlas a la formación permite que el estudiante aprenda tanto el contenido de un análisis como las condiciones que lo hacen verificable. No se trata de ocultar la infraestructura indefinidamente, sino de decidir qué complejidad abordar en cada momento de una secuencia docente.

En América Latina, las propuestas de computación transversal han planteado compartir protocolos, servicios y capacidades entre proyectos e instituciones [2, 3]. El término transversal se utiliza aquí para describir prácticas reutilizables más allá de un experimento: organizar código y datos, definir dependencias, ejecutar tareas, registrar resultados y comunicar cambios. La formación en física ofrece un contexto concreto para aprenderlas, mientras su utilidad puede extenderse a otras áreas científicas.

Este ensayo técnico relaciona repositorios, entornos declarativos y automatización con tres tareas docentes: preparar una práctica, revisar sus resultados y conservar el procedimiento. La contribución es una arquitectura verificable y una comparación de opciones de despliegue, acompañadas de un protocolo para medir tiempo, recursos e incidencias. Su objeto es el diseño de la formación en física; los resultados documentales y técnicos se distinguen de una evaluación del aprendizaje.

2. Antecedentes: acceso, infraestructura y continuidad

La experiencia venezolana con ATLAS Open Data documenta su uso en tesis y acompañamiento académico [4]. El despliegue multiusuario descrito por Sánchez Pineda y Guerrieri ofrece un antecedente publicado de acceso compartido a herramientas de análisis [5]. Ambos casos sitúan el problema docente en la relación entre recurso disponible, procedimiento ejecutable y apoyo para interpretar resultados. La propuesta de computación transversal de 2024 amplía esa perspectiva hacia capacidades compartidas entre instituciones [3].

El acceso mediante navegador también antecede a Codespaces. Una presentación del ROOT Workshop de 2015 relacionó análisis web de datos de ATLAS, tutoriales de ROOT y actividades de CEVALE2VE [6]. En una escala comunitaria, el artículo colectivo *Software Training in HEP* organiza competencias generales de software, herramientas específicas de física y técnicas avanzadas [7]. Esta distinción permite situar Git y la automatización junto a herramientas disciplinares, sin hacer depender la formación de una única plataforma.

ESCAPE vincula infraestructuras de física de partículas y astrofísica con servicios de ciencia abierta [8]. Su directorio incluye JupyterHub, entornos Docker, documentación de Rucio y recursos de ATLAS Open Data [9]. VRE Hub documenta un entorno virtual orientado a flujos de análisis y preservación conforme a principios FAIR [10]. Estas infraestructuras articulan servicios de investigación; un repositorio docente delimita un ejercicio y sus dependencias. La escala cambia, pero permanecen tres requisitos: acceso a los datos, descripción del software y conservación del procedimiento.

El aporte del presente ensayo es llevar esos requisitos al diseño de una sesión. La actividad debe definir qué se ejecuta, qué decisión científica toma el estudiante y qué evidencia queda para revisarla. La arquitectura siguiente separa esas responsabilidades y permite compararlas entre servicios.

Las lecciones de Software Carpentry sitúan automatización y control de versiones dentro de la formación práctica de investigadores [11]. Para los cuadernos, las reglas de Rule y colaboradores conectan organización del análisis, documentación y reproducibilidad [12]. En este diseño, el ejercicio debe conservar tanto su ejecución como la explicación de las decisiones científicas.

3. Metodología y arquitectura del entorno docente

3.1 Separar contenido, entorno y ejecución

El análisis técnico examina las interfaces entre contenido, entorno y ejecución, contrasta opciones con su documentación oficial y especifica qué registros permitirían evaluar una sesión. La figura 1 representa esas interfaces. La arquitectura distingue tres capas. La primera contiene el problema científico: datos de ejemplo, ejercicios, explicaciones y criterios de interpretación. La segunda define el software necesario para trabajar. La tercera ejecuta comprobaciones y produce salidas. Esta separación permite modificar una pregunta sin rediseñar el entorno, o actualizar una dependencia sin reescribir toda la actividad.

Los contenedores de desarrollo describen un entorno mediante archivos versionados; GitHub Codespaces puede utilizarlos para ofrecer un espacio accesible desde el navegador [13]. Esta descripción debe incluir dependencias y comandos de preparación. El acceso web reduce la necesidad de instalar herramientas científicas en cada equipo, pero sigue requiriendo una cuenta, conectividad y recursos disponibles. Un entorno descrito no es necesariamente inmutable: imágenes etiquetadas y paquetes sin fijar pueden cambiar entre ejecuciones.

Los repositorios docentes `jupyter-on-codespaces` y `exercises-silafae` proporcionan ejemplos públicos del enfoque [14]. Para una sesión, la unidad que se debe conservar es una revisión concreta del repositorio junto con la imagen del entorno y la versión del conjunto de datos. Así se distingue el material impartido de las actualizaciones posteriores.

3.2 Git y automatización como herramientas de razonamiento

Git registra cambios en los materiales y permite relacionar una decisión con una revisión concreta. En una actividad de física, un cambio puede afectar la selección de eventos, una normalización o una figura. El estudiante debería poder explicar qué modificó y por qué, además de obtener una salida. El historial ayuda a revisar esas decisiones, aunque no sustituye la explicación científica.

La automatización ejecuta tareas previamente definidas. Un flujo puede comprobar que un cuaderno corre desde un estado limpio o generar una figura con parámetros conocidos. Una configuración de integración continua complementa el entorno interactivo al ejecutar el mismo comando de validación sobre las fuentes versionadas. La migración a otro servicio debe probar ese comando y adaptar autenticación, almacenamiento y configuración del ejecutor.

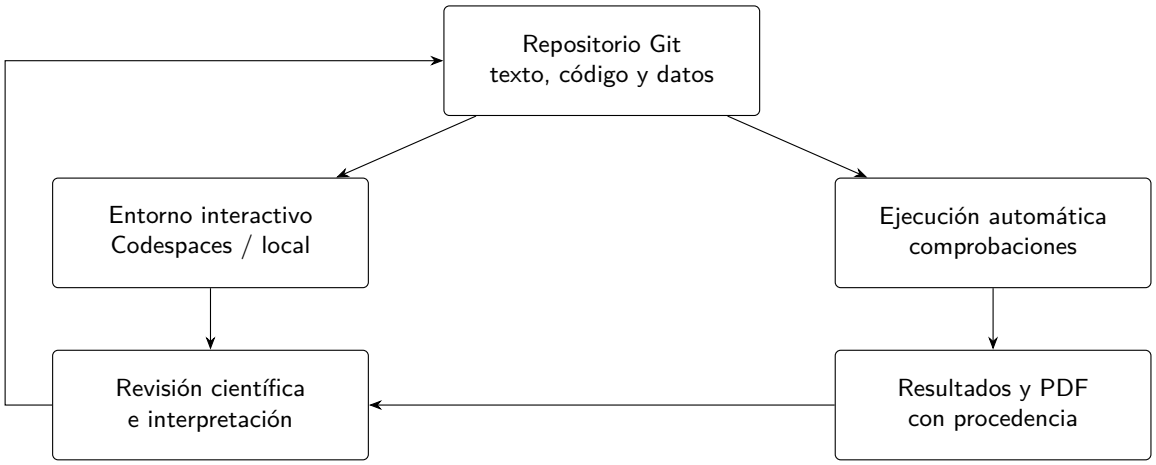


Figura 1: Arquitectura de referencia del trabajo docente. El entorno interactivo y la ejecución automática consumen fuentes compartidas; la revisión científica sigue siendo una tarea humana.

3.3 Una práctica de física que se pueda inspeccionar

Como diseño docente proponemos partir de una distribución física sencilla producida a partir de datos abiertos. Una secuencia puede pedir reproducir la distribución de referencia, variar una selección, comparar el resultado y justificar la diferencia. Deben declararse unidades, muestra utilizada y normalización. Una comprobación automática puede detectar una columna ausente o una ejecución incompleta; no puede establecer por sí sola si una elección física es apropiada.

Conviene conservar una muestra pequeña para comprobar el funcionamiento y separar esa prueba del análisis completo. También debe evitarse que un cuaderno conserve resultados de una ejecución anterior incompatibles con su código actual. Ejecutarlo desde un estado limpio ayuda a detectar dependencias implícitas. El producto evaluable es el análisis acompañado de una explicación de sus decisiones.

4. Resultados del análisis técnico

4.1 Opciones de despliegue y responsabilidades

El cuadro 1 compara decisiones operativas. Binder 2.0 describe la publicación de entornos interactivos a partir de repositorios [15]; el servicio público mybinder.org establece límites para sesiones efímeras [16]. JupyterHub permite administrar entornos por usuario y autenticación bajo control del operador [17]. Colab ofrece cuadernos gestionados con disponibilidad y límites variables [18]. Codespaces añade un entorno de desarrollo asociado al repositorio, con consumo de cómputo y almacenamiento facturable [19].

La comparación permite una decisión concreta: un ejercicio breve que solo necesita ejecutar un cuaderno tiene requisitos diferentes de un curso que enseña ramas, revisión de cambios y desarrollo de software. Codespaces encaja en este segundo objetivo si hay cuentas, conectividad y presupuesto; un JupyterHub institucional puede concentrar la administración cuando existe

Cuadro 1: Comparación operativa según documentación de las plataformas consultada en octubre de 2026.

Opción	Administración	Conservación del trabajo	Condición a planificar
Codespaces	Servicio gestionado; configuración del repositorio	Espacio persistente hasta su eliminación; cambios a Git	Cuentas, cuotas, cómputo y almacenamiento
Binder público	Servicio comunitario; entorno desde repositorio	Sesión efímera; exportar cambios	Límites de sesión y capacidad compartida
JupyterHub	Operador institucional o proveedor	Depende del almacenamiento configurado	Autenticación, capacidad, respaldo y soporte
Colab	Cuadernos gestionados	Cuaderno guardado; estado de ejecución temporal	Límites variables y dependencias adicionales

soporte local. Binder facilita demostraciones compartibles, mientras Colab ofrece una entrada rápida a cuadernos. La elección debe incluir el procedimiento para exportar el trabajo del alumno y conservarlo al terminar.

4.2 Contexto docente e inventario de evidencia

SILAFAE XV, celebrado en noviembre de 2024, constituye el primer contexto docente aquí considerado: el repositorio público exercises-silafae conserva ejercicios asociados al encuentro [14]. Los materiales permiten examinar la propuesta docente, pero su estado actual no identifica por sí solo la revisión empleada durante las sesiones. La 47.ª CERN School of Computing se celebró en Santiago del 11 al 24 de enero de 2026 [20]; su página de docentes incluye a Arturo Sánchez [21]. La presentación inaugural del 12 de enero informa de 69 participantes seleccionados, 20 nacionalidades y 44 institutos [22]. Son cifras de la escuela, no de asistencia a una sesión ni de éxito de incorporación. La misma presentación describe infraestructura remota proporcionada por CERN IT y trabajo por parejas; no demuestra que Codespaces fuera la plataforma general del curso. Estos registros sitúan el contexto formativo sin atribuir a la arquitectura propuesta un despliegue que la fuente no documenta. El cuadro 2 explicita qué evidencia sustenta cada afirmación del ensayo.

Cuadro 2: Objetos y evidencias disponibles. Las fuentes se identifican en el texto y en el registro de evidencia.

Objeto	Evidencia	Qué permite concluir
Materiales docentes	Repositorios públicos LAA-HECAP	Inspección de ejercicios y configuración actual
Contexto CSC 2026	Programa y lista de docentes de CERN	Fechas y participación docente
Diseño de infraestructura	Documentación de servicios	Distribución de responsabilidades y restricciones
Rendimiento de cohortes	Sin registros operativos en el corpus	Objeto de la evaluación propuesta

El resultado técnico de esta lectura es separar tres objetos que suelen confundirse: un repositorio disponible, un entorno que ejecuta una tarea y una experiencia docente evaluada. Cada uno

necesita una comprobación diferente. El repositorio permite inspección; una prueba limpia verifica ejecución; un registro por sesión permite estimar tiempos, incidencias y recursos. La arquitectura organiza esas comprobaciones sin sustituir una por otra.

5. Protocolo de evaluación y gestión de recursos

5.1 Tiempo y costo

La preparación del instructor y el inicio del estudiante deben medirse por separado. Para cada sesión se propone registrar un identificador anónimo, hora de inicio, primera ejecución válida, resultado de la comprobación y categoría de incidencia. El informe debe incluir participantes previstos, casos medidos, ejecuciones correctas y datos ausentes, junto con mediana y dispersión de los tiempos disponibles. Una preparación previa o una imagen precargada se declara como condición de la medición.

El registro económico distingue horas activas por tamaño de máquina, horas-núcleo, almacenamiento en GB-hora y uso de Actions para preconstrucciones. El almacenamiento sigue existiendo cuando el entorno está detenido; la tarifa y las cuotas aplicables se registran con su fecha [19]. El costo por participante requiere indicar si el denominador son inscritos, asistentes o personas que completaron la práctica. El repositorio adjunta plantillas vacías y un agregador que calcula consumos a partir de registros explícitos, sin asignar una tarifa universal ni reconstruir consumos históricos ausentes.

5.2 Fallos que el diseño debe contemplar

El cuadro 3 vincula riesgos operativos con señales y respuestas. Son escenarios de diseño, no incidentes atribuidos a una cohorte. Una prueba previa debe incluir cuenta de estudiante, permisos, arranque limpio y descarga de la muestra de datos. Con una alternativa local o un cuaderno exportable se puede continuar una parte de la actividad ante interrupciones del servicio.

Cuadro 3: Riesgos de diseño y respuesta propuesta; no es un registro de incidentes observados.

Riesgo	Señal a registrar	Respuesta prevista
Acceso o permisos	Inicio rechazado; categoría del error	Probar cuentas y acceso antes de clase
Cuota o presupuesto	Aprovisionamiento bloqueado	Revisar límites y responsable de facturación
Conectividad	Descarga incompleta o sesión interrumpida	Muestra reducida y materiales descargables
Dependencia incompatible	Validación fallida tras actualizar	Imagen identificada y prueba desde estado limpio
Pérdida de trabajo	Entorno eliminado sin exportación	Guardado en Git y copia de resultados

La evaluación del aprendizaje necesita una tarea adicional: interpretar una figura, justificar una selección o reproducir un análisis días después. El tiempo de aprovisionamiento mide acceso al entorno; la explicación del resultado mide otra dimensión. Un diseño comparativo debería mantener tarea y condiciones equivalentes y registrar experiencia previa, conectividad y apoyo recibido. El repositorio de estos manuscritos aplica el mismo patrón a una tarea editorial: fuentes LaTeX versionadas, comprobaciones automáticas y PDF actualizados en rutas estables. Su código

y sus manifiestos están disponibles en `https://github.com/artfisica/acfiman-open-science-papers` como ejemplo ejecutable de conservación del procedimiento.

6. Discusión

La contribución de un entorno declarativo es hacer explícitas las dependencias y ofrecer un punto de partida compartido. En formación en física, ese punto de partida permite dedicar una secuencia docente a decisiones científicas: selección de datos, unidades, normalización e interpretación. Enseñar Git y automatización añade un registro de decisiones que puede revisarse con el estudiante y reutilizarse en otras disciplinas.

El servicio remoto redistribuye trabajo y costos. Parte de la instalación pasa al equipo docente; cuentas, cuotas y conectividad siguen condicionando la sesión. Una imagen fijada por digest identifica el software distribuido, pero datos externos y servicios requieren su propia conservación. El mantenimiento debe combinar una versión estable para cada curso con una ruta explícita de actualización y prueba.

Alcance y límites. El trabajo es un ensayo técnico basado en documentación y diseño de arquitectura. No dispone de registros individuales de incorporación, exportaciones de consumo ni evaluación causal del aprendizaje de las escuelas citadas. Por ello no informa cifras históricas de velocidad, costos o incidencia de fallos. La comparación de plataformas describe responsabilidades y condiciones documentadas, no un ensayo de rendimiento; las plantillas proponen cómo obtener esa evidencia en futuras sesiones.

7. Conclusiones

La computación transversal se concreta en prácticas que sobreviven al cambio de plataforma: describir dependencias, versionar procedimientos, ejecutar comprobaciones y conservar resultados interpretables. La comparación identifica dónde recae la administración y qué debe prever una actividad de física para compartir un entorno de trabajo.

La evaluación futura debe reunir tres tipos de evidencia: ejecución limpia del análisis, consumo y soporte de la sesión, y comprensión del estudiante. Mantenerlos separados permite mejorar la infraestructura sin confundir rapidez de acceso con aprendizaje. Las plantillas de registro y el ejemplo editorial ofrecen instrumentos concretos para iniciar esa evaluación.

Vínculos profesionales

Arturo Sánchez Pineda mantiene una relación profesional con INAIT SA y participa en iniciativas de formación y ciencia abierta citadas en este trabajo. Las afiliaciones de este manuscrito son Creative Commons Venezuela y Creative Commons Switzerland. [**Revisión editorial: La declaración de financiación específica se completará antes del envío.**]

Referencias

[1] Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L. y Teal, T.K. Good enough practices in scientific computing. *PLoS Computational Biology* **13**(6), e1005510 (2017). https://doi.org/10.1371/journal.pcbi.1005510.

[2] Sánchez Pineda, A. A proposal for Transversal Computer-related Strategies & Services for Scientific and Training efforts for the LASF4RI. arXiv:2104.09622 (2021). https://doi.org/10.48550/arXiv.2104.09622.

[3] Sánchez Pineda, A. An empirical case and ongoing efforts on Transversal Computer Strategies & Services for Scientific and Training efforts for the LASF4RI. Zenodo, v3 (2024). https://doi.org/10.5281/zenodo.13324266.

[4] Sánchez Pineda, A. Boosting high-energy physics education around the world with ATLAS Open Data. ATLAS/CERN (2018). https://atlas.cern/updates/blog/HEP-education-worldwide-with-atlas-open-data.

[5] Sánchez Pineda, A. y Guerrieri, G. A proposal for Open Access data and tools multi-user deployment using ATLAS Open Data for Education. *EPJ Web of Conferences* **251**, 01008 (2021). https://doi.org/10.1051/epjconf/202125101008.

[6] Sánchez Pineda, A. Web and Grid based ROOT analysis, and national ROOT analysis tutorials in ATLAS. ROOT Workshop, 15 de septiembre de 2015. Presentación. https://indico.cern.ch/event/349459/contributions/822780/attachments/1154438/1658747/ROOT_Workshop_Arturo_Sanchez_09152015.pdf.

[7] Malik, S., Meehan, S., Lieret, K. y colaboradores. Software Training in HEP. *Computing and Software for Big Science* **5**, 22 (2021). https://doi.org/10.1007/s41781-021-00069-9. Versión de autor: https://arxiv.org/abs/2103.00659.

[8] ESCAPE. European Science Cluster of Astronomy & Particle physics ESFRI research infrastructures. Sitio institucional. Consulta: 21 de octubre de 2026. https://projectescape.eu/.

[9] ESCAPE. ESCAPE tools suite. Directorio de recursos técnicos. Consulta: 21 de octubre de 2026. https://escape2020.pages.in2p3.fr/virtual-environment/tools/.

[10] VRE Hub. Virtual Research Environment. Descripción del proyecto y repositorios. Consulta: 21 de octubre de 2026. https://github.com/vre-hub.

[11] Wilson, G. Software Carpentry: lessons learned. Versión 2. *F1000Research* **3**, 62 (2016). https://doi.org/10.12688/f1000research.3-62.v2.

[12] Rule, A., Birmingham, A., Zuniga, C. y colaboradores. Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLoS Computational Biology* **15**(7), e1007007 (2019). https://doi.org/10.1371/journal.pcbi.1007007.

[13] GitHub. Introduction to dev containers. Documentación técnica, consulta: 21 de octubre de 2026. https://docs.github.com/en/codespaces/setting-up-your-project-for-codespaces/adding-a-dev-container-configuration/introduction-to-dev-containers.

[14] LAA-HECAP. Repositorios docentes jupyter-on-codespaces y exercises-silafae. Consulta: 21 de octubre de 2026. https://github.com/laa-hecap/jupyter-on-codespaces; https://github.com/laa-hecap/exercises-silafae.

[15] Project Jupyter y colaboradores. Binder 2.0—Reproducible, Interactive, Sharable Environments for Science at Scale. *Proceedings of the 17th Python in Science Conference* (2018). https://doi.org/10.25080/Majora-4af1f417-011.

[16] Project Jupyter. Binder: Usage guidelines. Consulta: 21 de octubre de 2026. https://mybinder.readthedocs.io/en/latest/about/user-guidelines.html.

[17] Project Jupyter. JupyterHub documentation. Consulta: 21 de octubre de 2026. https://jupyterhub.readthedocs.io/en/stable/.

[18] Google. Colaboratory: Frequently Asked Questions. Consulta: 21 de octubre de 2026. https://research.google.com/colaboratory/faq.html.

[19] GitHub. GitHub Codespaces billing. Consulta: 21 de octubre de 2026. https://docs.github.com/en/b

illing/concepts/product-billing/github-codespaces.

[20] CERN. 47th CERN School of Computing – Latin America 2026, 11–24 de enero de 2026. Programa y materiales. https://indico.cern.ch/event/1551289/timetable/?view=standard_numbered.

[21] CERN. 47th CERN School of Computing – Latin America 2026: Lecturers. Perfil de Arturo Sánchez. Consulta: 21 de octubre de 2026. https://indico.cern.ch/event/1551289/page/39511-lecturers.

[22] CERN School of Computing. Presentación inaugural de la dirección, Santiago, 12 de enero de 2026. Diapositivas 42 y 51. https://indico.cern.ch/event/1551289/contributions/6608998/attachments/3198829/5694384/2026-01-12-Santiago-Director-Slides-Opening-v05.pdf.